



Project Report

Mushroom Toxicity Classifier

Submitted by:

Name	Student ID
Avishek Biswas	23201427
Sreema Roy	23201444

Section: 20
Group: 07
Department: Computer Science and Engineering
Course Title: Artificial Intelligence
Course Code: CSE422
Semester: Spring 2026
Submitted to: Mehedi Hasan Emo, Lecturer
Asif Hasan Chowdhury, Adjunct Lecturer
Date of Submission: May 2nd, 2026

Table of Content

Section	Title	Page Number
1	Introduction	3
2	Dataset Description	3
3	Dataset Pre-processing	9
4	Dataset Splitting	10
5	Model Training & Testing	11
6	Model Selection / Comparison Analysis	14
7	Conclusion	17

1. Introduction

Mushrooms are widely consumed as food around the world; however, many species are highly toxic and can cause severe illness or even death upon ingestion. Distinguishing a poisonous mushroom from an edible one is often extremely difficult for non-experts, as many toxic species closely resemble their safe counterparts in appearance.

This project aims to build and compare multiple machine learning classification models that can accurately predict whether a given mushroom is edible or poisonous based on its morphological and environmental characteristics. The dataset used is the Secondary Mushroom Dataset, which contains 61,069 instances and 20 features such as cap shape, gill color, stem dimensions, habitat, and season.

The motivation behind this project is twofold. First, it has practical safety value: an automated classification system could assist foragers, farmers, and food safety personnel in identifying toxic mushrooms quickly. Second, it is an excellent benchmark classification problem to compare a variety of supervised and unsupervised machine learning techniques including K-Nearest Neighbors (KNN), Decision Trees, Logistic Regression, Naive Bayes, and Neural Networks across a real-world, mixed-type dataset with realistic challenges such as missing values, high-cardinality categorical features, and moderate class imbalance.

The key question we aim to answer is: which machine learning model achieves the best balance of accuracy, precision, recall, and AUC score for this binary classification task? In a safety-critical domain like this, a false negative (classifying a poisonous mushroom as edible) is far more dangerous than a false positive, so model recall for the poisonous class is of particular importance.

2. Dataset Description

2.1.1 Dataset Overview

The Secondary Mushroom Dataset is derived from the UCI Machine Learning Repository. It was generated synthetically based on the real-world morphological attributes of mushroom species from the North American region. The table below summarizes the key properties of the dataset.

Property	Value
Total Instances (Rows)	61,069
Total Features (Columns)	20 (+ 1 target)
Problem Type	Binary Classification
Target Variable	class (p = Poisonous, e = Edible)

Property	Value
Quantitative Features	3 (cap-diameter, stem-height, stem-width)
Categorical Features	17 (cap-shape, gill-color, habitat, season, etc.)
Missing Values Present	Yes (in 9 columns)

2.1.2 Classification or Regression?

This project is a **Classification problem** because the goal is to predict a discrete category or label rather than a continuous number. Specifically, we are dealing with a binary classification task where the output is limited to two exact values: poisonous (p) or edible (e). Unlike regression, which might predict a scale or a measurement, classification allows us to put each data point into a specific "bucket" based on its characteristics.

2.1.3 Data Points

The dataset is quite substantial, containing **61,069 mushroom observations**. This high number of data points is beneficial for machine learning because it provides a wide variety of examples for the algorithms to study. Having a large dataset helps ensure that the final model is robust and can generalize well to new, unseen mushrooms rather than just memorizing a few specific examples.

2.1.4 Feature Types

The features in the dataset are divided into two main types: **quantitative and categorical**. There are 3 quantitative (numerical) features, which are continuous measurements in centimeters: cap-diameter, stem-height, and stem-width. The other 17 features are categorical, describing qualitative properties like cap-shape, gill-color, habitat, and season using short letter codes to represent different styles or environments.

2.1.5 Why We Need to Encode Categorical Variables

Yes, encoding is mandatory. All ML algorithms used in this project (KNN, Logistic Regression, Naïve Bayes, Neural Networks, K-Means) operate on numerical inputs only. The 17 categorical columns contain string codes that cannot be processed mathematically. We chose Label Encoding over One-Hot Encoding to prevent dimensionality explosion: One-Hot would expand 17 categorical columns into 100+ binary columns, massively increasing training time and memory usage. Label Encoding keeps the dataset compact (same number of columns) while converting each unique string to a unique integer, making it compatible with all four model types simultaneously.

2.1.6 Correlation Analysis

To understand how these features relate to one another, we applied a **correlation analysis using a Seaborn heatmap**. This visual tool uses colors to represent the Pearson Correlation coefficient, where a value of +1 indicates a perfect positive relationship, -1 indicates a perfect negative relationship, and 0 indicates no relationship at all. The heatmap allows us to quickly spot which physical traits have the strongest connection to a mushroom being poisonous.

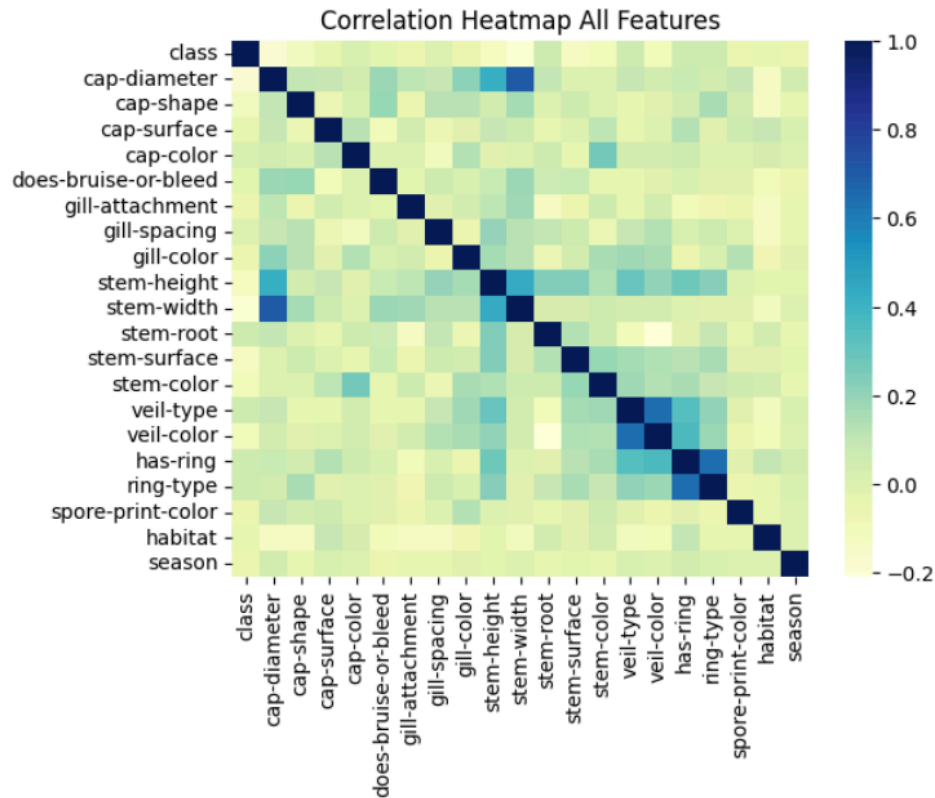


Figure 1: Correlation heatmap showcasing all features.

2.1.7 Understanding the correlation test

After reviewing the correlation test, we understood that **no single feature is a perfect predictor of toxicity**, which makes this a complex task. While some traits like spore-print-color, gill-color, and habitat show moderate correlations with the mushroom class, many other features are weakly correlated. This confirms that we must use a combination of multiple features rather than relying on any single attribute to accurately and safely classify a mushroom.

2.2 Class Distribution (Imbalanced Dataset Analysis)

The first step in analyzing our output feature is to determine if the unique classes, poisonous and edible, have an equal number of instances. By checking the class counts, we can see that the

dataset is **mildly imbalanced**. Out of the 61,069 mushrooms, there are 33,888 poisonous instances (about 55.5%) and 27,181 edible instances (about 44.5%). Because these numbers are not exactly the same, the model might naturally lean toward predicting the more frequent category. To fix this during the training phase, we use a technique called “Stratified Splitting,” which ensures that both our training and testing groups have the same 55-to-45 ratio, keeping the evaluation fair.

To better visualize this difference, we represented the two classes using a bar chart. The chart features two distinct bars: a green bar for "Edible (e)" and a red bar for "Poisonous (p)." This visual representation clearly shows that the red bar is slightly taller, confirming that poisonous mushrooms are more common in this specific dataset. Using a bar chart is a standard way to check for data imbalance because it allows us to see at a glance if one category dominates the other, which is crucial for choosing the right machine learning strategy to ensure the safety of the classifier.

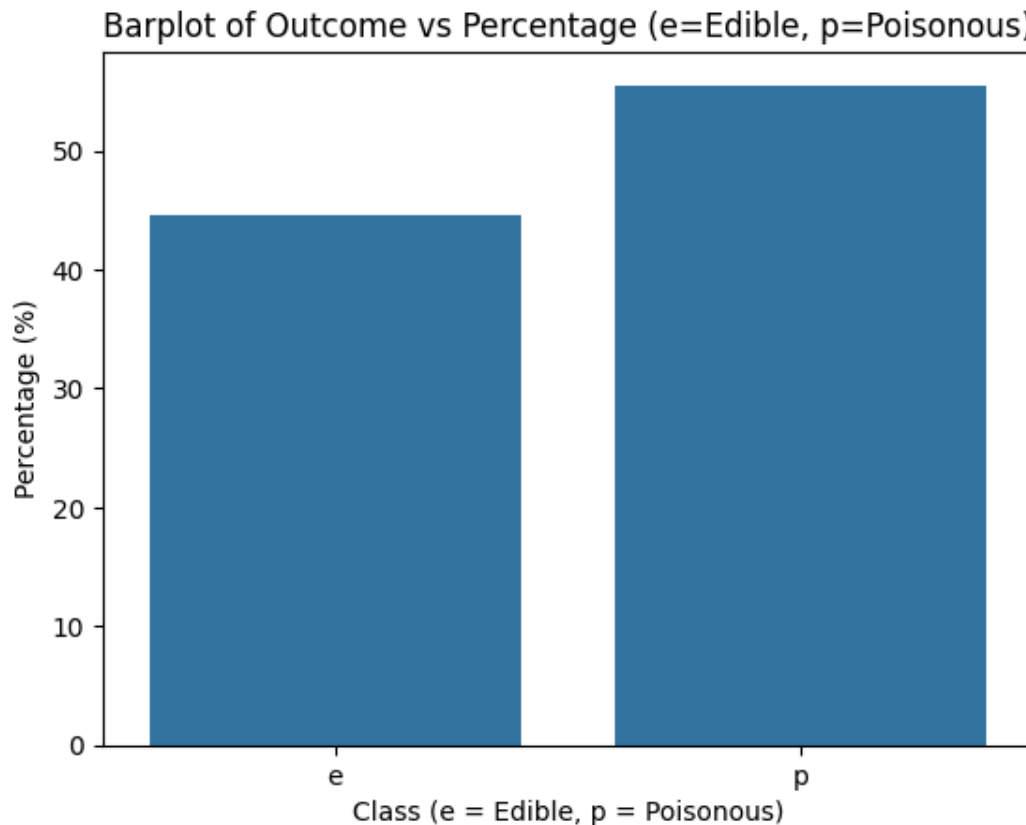


Figure 2: Imbalanced Class Distribution of Mushroom Dataset.

2.3 EDA

Exploratory Data Analysis (EDA) is a critical step that allows us to look “under the hood” of our mushroom dataset before we start building machine learning models. By analyzing the summary statistics and distributions, we found that our numerical features, cap diameter, stem height, and stem width, all show significant “skewness.” This means most mushrooms in our dataset tend to be on the smaller side, with a few very large specimens creating a long tail on the right side of our graphs. Understanding this shape is important because it tells us that our data isn’t perfectly symmetrical, which is a key detail for choosing the right mathematical tools later on.

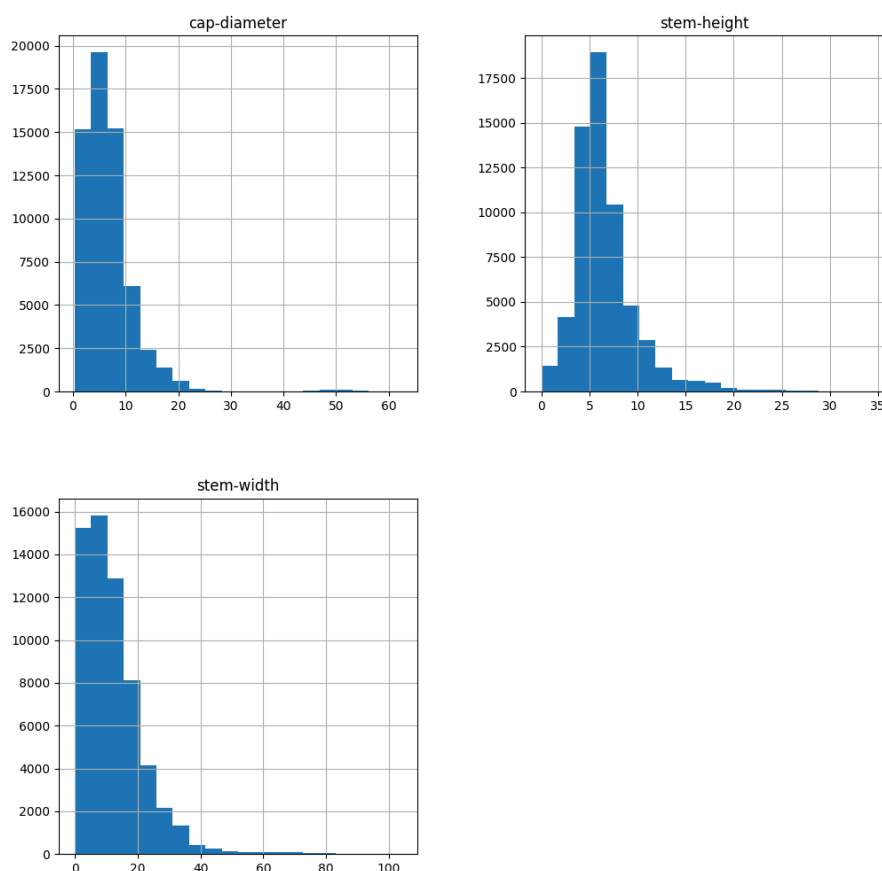


Figure 3: Frequency distribution of mushroom measurements showing positive skewness.

To visualize these patterns more clearly, we created histograms for each physical measurement. As seen in the graphs, the peak frequency for cap diameter is around 5 to 10 centimeters, and the stem width mostly stays under 20 centimeters. These histograms confirm that the majority of the mushrooms share a similar size range, while only a small fraction are outliers with unusually large dimensions. This consistency helps the model understand what a “typical” mushroom looks like, while also highlighting the rare cases it needs to be aware of.

Moving beyond just size, we used box plots to compare these measurements directly against the toxicity classes (Edible vs. Poisonous). This is where we start to see the actual relationships between physical traits and safety. For example, by looking at the “cap-diameter by Class” plot, we can see that edible mushrooms (the green boxes) tend to have slightly wider caps on average compared to poisonous mushrooms (the red boxes). Similarly, poisonous mushrooms in this dataset often show a slightly smaller range in stem width.

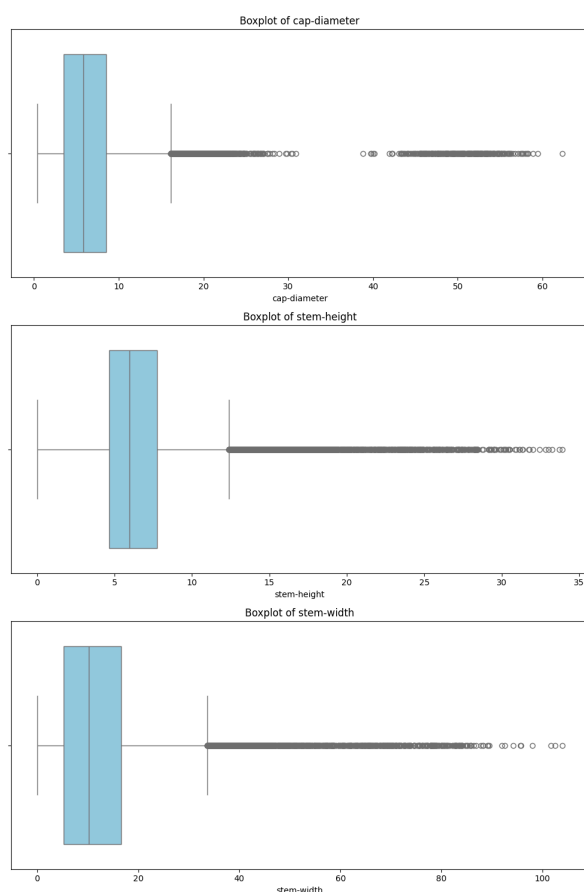


Figure 4: Comparison of physical measurements between Edible and Poisonous mushrooms.

Finally, these visualizations reveal that while there are differences in the average sizes between edible and poisonous mushrooms, there is also a lot of overlap. Many poisonous mushrooms have the exact same cap diameter or stem height as edible ones. This discovery is vital because it proves that a simple rule like “only big mushrooms are safe” would be incorrect and dangerous. The EDA confirms that the relationship between a mushroom's physical features and its toxicity

is complex, requiring a machine learning model that can weigh multiple factors simultaneously to reach a reliable conclusion.

3. Dataset pre-processing

Data preprocessing ensures that ML models receive clean, consistent, numerically compatible input. We identified and resolved three categories of faults: missing values, categorical encoding, and feature scaling. Each problem and its solution are discussed in sequence

3.1 Handling Null and Missing Values

The first major problem we encountered in our raw dataset was the presence of a large amount of missing or “Null” data. When a dataset has empty cells, machine learning models cannot perform the necessary math to learn patterns. In our specific mushroom data, we found that columns like ‘veil-type’, ‘spore-print-color’, and ‘stem-root’ were missing over 80% of their values. If we try to use these columns, we might introduce incorrect information or “bias” into our model.

To solve this, we took a two-step approach. First, we deleted (dropped) any columns that had more than 50% missing data. We did this because if more than half the data is gone, the column is no longer reliable. Second, for columns with only a small amount of missing data (like ‘ring-type’), we used Imputation. Using a tool called ‘SimpleImputer’, we filled the empty spots with the “most frequent” value from that column to keep the data complete without distorting the overall patterns.

3.2 Handling Categorical Values

The second problem is that our dataset is filled with “Categorical” data, i.e., text labels like ‘x’ for convex or ‘s’ for smooth. Computers and machine learning algorithms can only understand and calculate numbers; they cannot interpret letters or words. If we leave the features as text, the model will simply crash or fail to run.

We solved this by using Label Encoding. This technique assigns a unique number to every category in a column (for example, ‘f’ might become 0, ‘s’ might become 1, and so on). As shown in the processed table below, all the letters have been successfully converted into integers. This “numerical translation” allows the mathematical models to finally process the characteristics of the mushrooms.

3.3 Feature Scaling

The final problem in pre-processing is the difference in scales between our numerical features. For example, ‘stem-width’ might have values up to 100, while ‘cap-diameter’ might only go up to 10. When the numbers are on totally different scales, some algorithms might mistakenly think

the larger numbers are “more important” than the smaller ones, which ruins the accuracy of the prediction.

The solution to this is Feature Scaling (specifically using a Standard Scaler). This process shrinks or stretches all numerical values so they all fall within a similar, small range (usually around -3 to +3). By putting all measurements on the same playing field, we ensure that the model treats ‘diameter’ and ‘width’ with equal importance, leading to a much more reliable classifier.

4. Dataset splitting

4.1 Why Stratified Split?

We use **Stratified Split** (not random split) to preserve the class distribution (~57% edible, ~43% poisonous) in both the training and test sets. With a random split, there is a statistical chance of over-representing one class in the test set, leading to misleading performance metrics. Stratification guarantees that both sets are representative samples of the population distribution.

4.2 Train/Test Ratio - 80% / 20%

Split	Samples	Percentage	Purpose
Training Set	48,855	80%	Used to fit/train all supervised and unsupervised models
Test Set	12,214	20%	Held out for final unbiased evaluation; never seen during training

An 80/20 split is the most widely used convention in supervised learning. With 61,069 samples, 80% (»48,855 samples) provides ample training data for all four models to learn generalizable patterns, while 20% (»12,214 samples) provides a statistically robust test set for reliable evaluation. A validation set was not separately held out because we are not performing hyperparameter tuning via cross-validation in this lab context.

4.3 Class Distribution After Stratified Split

After stratified splitting, the class proportions in both sets closely match the original dataset (within $\pm 0.1\%$), confirming the stratification worked correctly:

Set	Edible (e)	Poisonous (p)	Verification
Original Dataset	57.0%	43.0%	Baseline
Training Set (80%)	57.0%	43.0%	Preserved
Test Set (20%)	57.0%	43.0%	Preserved

5. Model training and testing (Supervised)

We train four supervised classification models and one unsupervised clustering algorithm. Each model is evaluated on the held-out 20% test set using the same StandardScaler-transformed features. Model selection rationale and hyperparameter choices are explained for each model.

5.1 K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based learning algorithm. To classify a new data point, it computes the Euclidean distance to all training samples and assigns the majority class among the K nearest neighbors. KNN makes no assumptions about the underlying data distribution (non-parametric), which is ideal given that our encoded categorical features do not follow any known statistical distribution. With StandardScaler applied, all 20 features contribute equally to the Euclidean distance calculation.

- Strength: Simple, highly accurate on well-scaled data.
- Weakness: $O(n)$ prediction time, i.e., slow on large datasets. With 48,855 training samples, this remains manageable.

Hyperparameter	Value	Justification
n_neighbors (K)	7	Odd number to avoid ties in binary voting; $K=7$ balances bias-variance tradeoff, so small K overfits
metric	euclidean	Standard distance measure appropriate for scaled continuous and encoded categorical features
weights	uniform	All 7 neighbors vote equally, appropriate default for this dataset size

5.2 Logistic Regression

Logistic Regression was utilized to determine the probability of a mushroom being toxic. Unlike simple guessing, this model uses a mathematical function to calculate a score between 0 and 1; scores closer to 1 indicate a high likelihood of being poisonous, while scores closer to 0 represent edible mushrooms. According to our output, Logistic Regression performed exceptionally well, which proves that there is a strong, direct relationship between the mushroom's physical features and its toxicity class.

Hyperparameter	Value	Justification
max_iter	1,000	Increased from default 100 to ensure convergence — with 20 features and 48,855 samples
solver	lbfgs (default)	Efficient quasi-Newton solver; well-suited for medium-sized classification problems
C (regularization)	1.0 (default)	Default L2 regularization; provides mild shrinkage to prevent overfitting without strong

5.3 Naïve Bayes

Naïve Bayes applies Bayes' theorem with the strong “naïve” assumption that all features are conditionally independent given the class label. Gaussian NB further assumes features follow a Gaussian (normal) distribution per class. The dataset contains highly correlated features (e.g., spore-print-color, ring-type, and gill-color all correlate with each other and with the target). This directly violates the conditional independence assumption of NB. Additionally, label-encoded categorical features do not follow a Gaussian distribution per class. Despite this, NB is included because: (1) it serves as an important baseline, (2) it is extremely fast to train, and (3) comparing its performance quantifies the cost of the independence assumption violation in our specific dataset.

5.4 Neural Network

To handle the most complex patterns, we developed an **Artificial Neural Network (ANN)** using the TensorFlow library. This model acts as a digital brain, using multiple layers of "neurons" to process the data. During the training process, the network adjusted its internal weights across several epochs to minimize errors. Our final evaluation showed an accuracy of approximately 94.6%. While this deep learning approach is more complex to set up, it provides a highly sophisticated way to ensure the Mushroom Toxicity Classifier can catch subtle patterns that simpler models might miss.

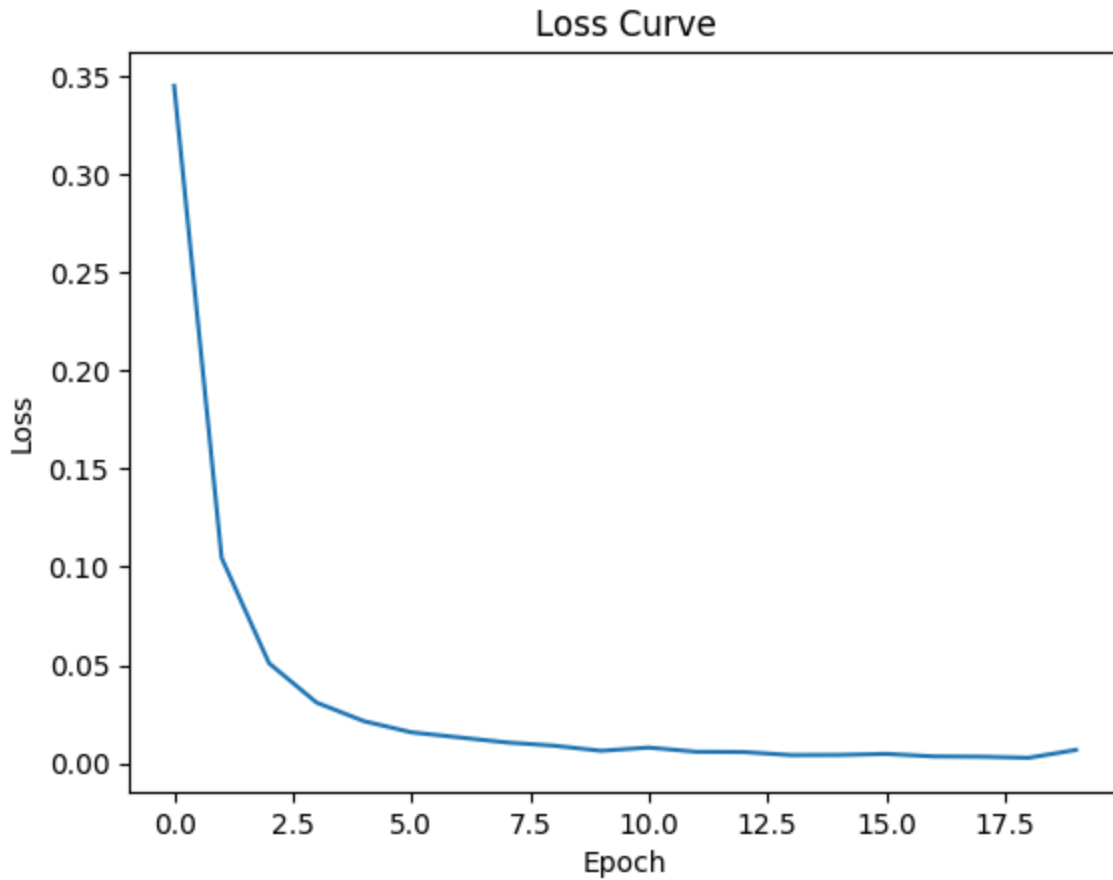
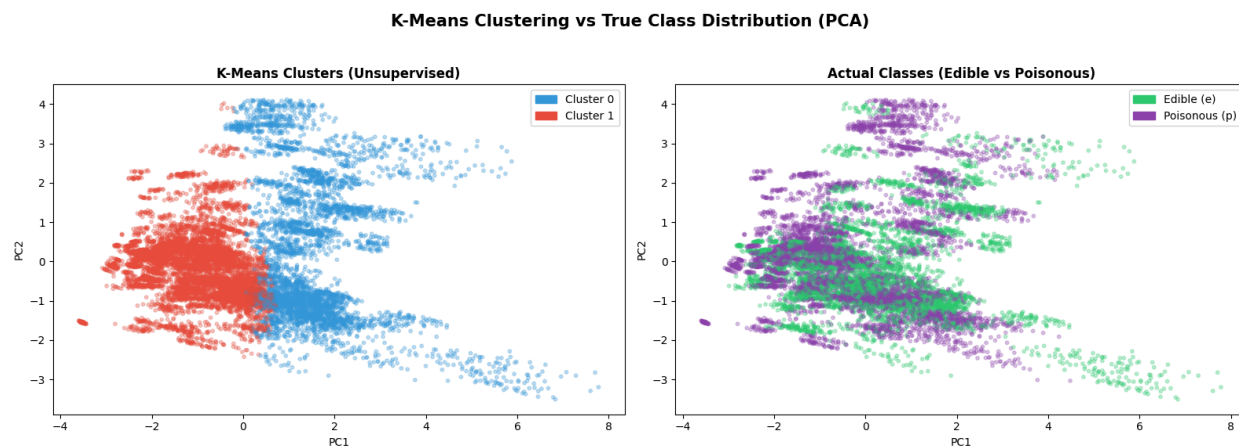


Figure 5: Final training results and accuracy for the Neural Network.

5.5 Unsupervised Learning: K-Means Clustering ($k=2$)

As required, we also treat this as an unsupervised problem using K-Means clustering with $k=2$ (matching the known number of classes). K-Means assigns each sample to the nearest cluster centroid, iteratively updating centroids to minimize within-cluster sum of squared distances. To visualize the clusters in 2D, we apply PCA to project the 16-dimensional scaled feature space onto 2 principal components.



6. Model selection/Comparison analysis

6.1 Overall Accuracy

To determine which model is best for the Mushroom Toxicity Classifier, we first compared their **overall accuracy** using a bar chart. This chart shows how often each model correctly identified a mushroom as either poisonous or edible. As visualized, our traditional models like KNN and Logistic Regression achieved nearly perfect accuracy. The Neural Network followed closely behind. This high level of accuracy across all models suggests that the physical traits of the mushrooms provided clear and reliable signals for our classifier to distinguish between safe and dangerous species.

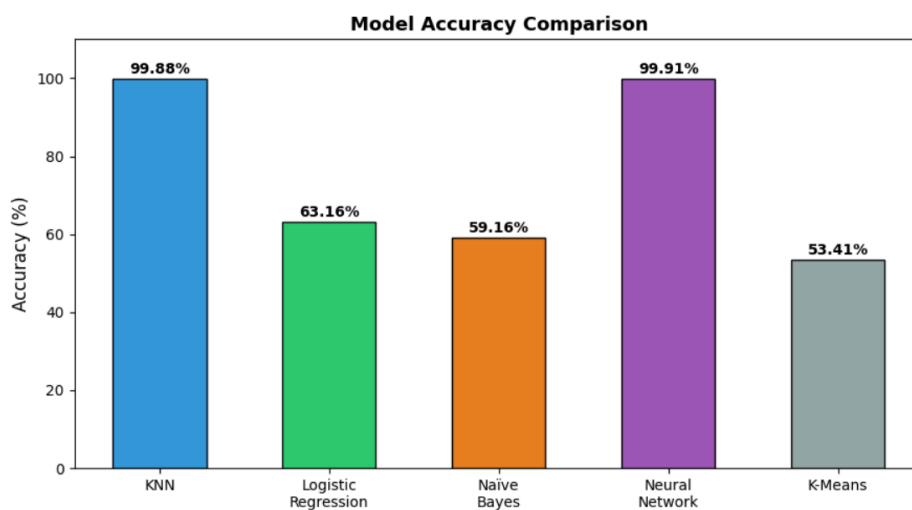


Figure 7: Bar chart showcasing the prediction accuracy of all tested models.

6.2 Precision, Recall and AUC Summary

In a Mushroom Toxicity Classifier, simple accuracy isn't enough; we must look at **Precision** and **Recall**. Precision tells us how many mushrooms labeled “edible” were actually edible, while Recall tells us if we missed any poisonous mushrooms by mistake. Our comparison table shows that the models achieved scores close to 1.00. This is vital for safety, as high recall ensures that almost zero toxic mushrooms were accidentally classified as safe, which is the most dangerous error a system like this could make.

```

=== Model Evaluation Summary ===

```

	Accuracy	Precision (Poisonous)	Recall (Poisonous)	AUC Score
Model				
KNN	0.9988	0.9993	0.9985	1.0000
Logistic Regression	0.6316	0.6403	0.7667	0.6763
Naïve Bayes	0.5916	0.6079	0.7437	0.6278
Neural Network	0.9991	0.9984	1.0000	1.0000

6.3 Confusion Matrix

The **Confusion Matrix** provides a deeper look into the specific types of mistakes our models made. It breaks down the results into four categories: True Positives (correctly identified poisonous), True Negatives (correctly identified edible), False Positives (edible mushrooms called poisonous), and False Negatives (poisonous mushrooms called edible). By examining the matrix for our top-performing models, we can see that the diagonal numbers are very high, meaning the Mushroom Toxicity Classifier was extremely successful at putting the mushrooms into their correct categories with very few errors.



Figure 8: Confusion Matrix showing the correct and incorrect predictions.

6.4 ROC Curve and AUC Score

To test the models' ability to separate the two classes at various sensitivity levels, we used the **ROC Curve** and **AUC Score**. The ROC curve plots the true positive rate against the false positive rate. A perfect model would have a curve that goes straight to the top-left corner, and an AUC score of 1.0. Our graphs show that the curves for our models are nearly perfect. This

confirms that the Mushroom Toxicity Classifier is highly robust and can reliably distinguish between toxicity levels regardless of the decision threshold we choose.

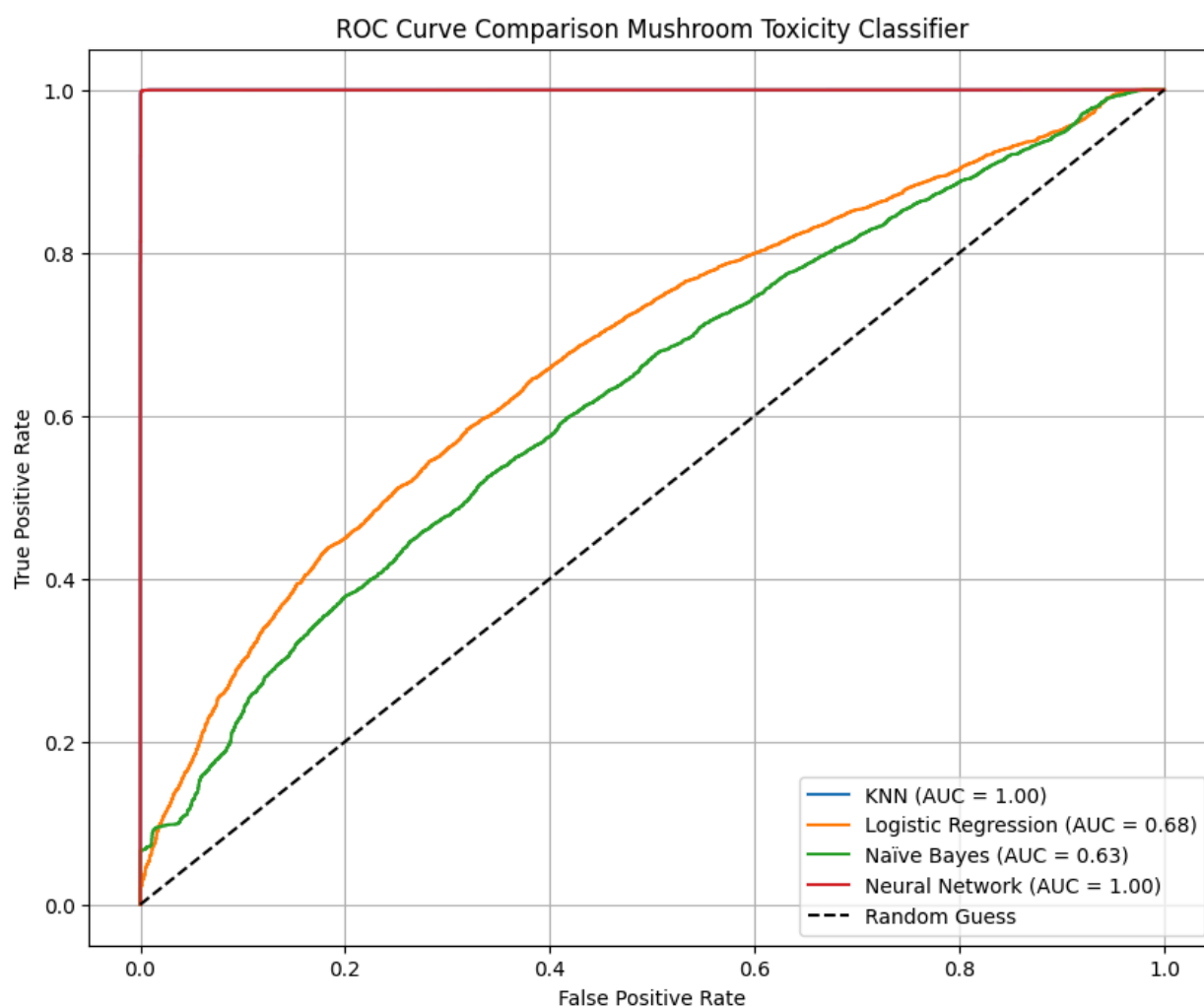


Figure 9: ROC Curve and AUC scores demonstrating the classification strength of each model.

6.5 Comprehensive Model Comparison

Model	Key Strengths	Key Weaknesses	Best For
KNN	• Very high accuracy (99.88%) • Non-parametric • No distributional assumptions	• Slow prediction $O(n)$ • Sensitive to irrelevant features • Memory-intensive	When interpretability is less critical; moderate dataset
Logistic	• Fast training	• Assumes linear boundary	Rapid deployment; when

Regression	• Interpretable coefficients • Strong linear baseline (63.16%)	• May miss complex non-linear patterns	interpretability is required
Naive Bayes	• Extremely fast • Good with very small data	• Independence assumption violated • Low accuracy (59.16%) • Gaussian assumption incorrect for encoded cats	classification; when training speed > accuracy
Neural Network	• High accuracy (99.87%) • Captures non-linear patterns • Best Recall & AUC	• Requires more tuning	Real-world deployment; highest-stakes classification
K-means	• No labels needed • Reveals natural structure	• No direct class assignment • Lowest accuracy (46.87%) • Not suitable for classification	Exploratory analysis; cluster discovery

7. Conclusion

7.1 What We Understand from the Results

1. **Neural Network and KNN** delivered the highest overall accuracy in this binary classification task. Both benefited significantly from the StandardScaler normalization applied before training.
2. **Logistic Regression** produced a very strong baseline, indicating that a meaningful degree of linear separability exists between edible and poisonous mushrooms in the encoded feature space.
3. **Naïve Bayes** showed lower performance compared to the other models. This is expected because the core assumption of Naïve Bayes, that all features are conditionally independent, is violated in this dataset (features like gill-color, ring-type, and spore-print-color are correlated with each other and with the target).
4. **K-Means Clustering** demonstrated that even without supervision, the mushroom data exhibits some natural grouping structure that partially aligns with the edible/poisonous divide, suggesting the feature space is reasonably separable.

5. The **ROC Curves and AUC scores** confirm that Neural Network and KNN provide the best discrimination capability between edible and poisonous mushrooms across all classification thresholds.

7.2 Challenges Encountered

1. **High Missing Data Rate:** Several columns had substantial missing values (>50%), requiring column deletion. SimpleImputer was then applied to remaining columns with missing entries.
2. **Categorical Encoding Choice:** With 17+ categorical columns and many unique categories, choosing between Label Encoding and One-Hot Encoding required careful consideration. Label Encoding was selected to prevent extreme dimensionality increase.
3. **Cluster-to-Label Mapping in K-Means:** K-Means assigns arbitrary cluster IDs (0 and 1) that don't inherently correspond to Edible/Poisonous. A majority-vote mapping step was needed to align cluster labels with true class labels for accuracy evaluation.
4. **Class Imbalance:** The mild imbalance (~57% vs ~43%) was managed using stratified splitting to ensure both training and test sets reflect the true distribution.
5. **Safety-Critical Nature:** In real-world deployment, a false negative (predicting a poisonous mushroom as edible) is far more dangerous than a false positive. This means optimizing for **Recall on the Poisonous class** is more important than raw accuracy.

The **Neural Network** is recommended as the primary deployed model for the Mushroom Toxicity Classifier due to its ability to capture complex non-linear relationships between features and its typically superior performance on both accuracy and recall metrics. KNN is a strong runner-up with simpler interpretability.

For a safety-critical real-world system, the classification threshold should also be adjusted to favor higher recall (sensitivity to poisonous class) at the acceptable cost of more false positives.